



© Karol Wisniwski



Engineer.wka@gmail.com



+48 795 738 507

 LinkedIn



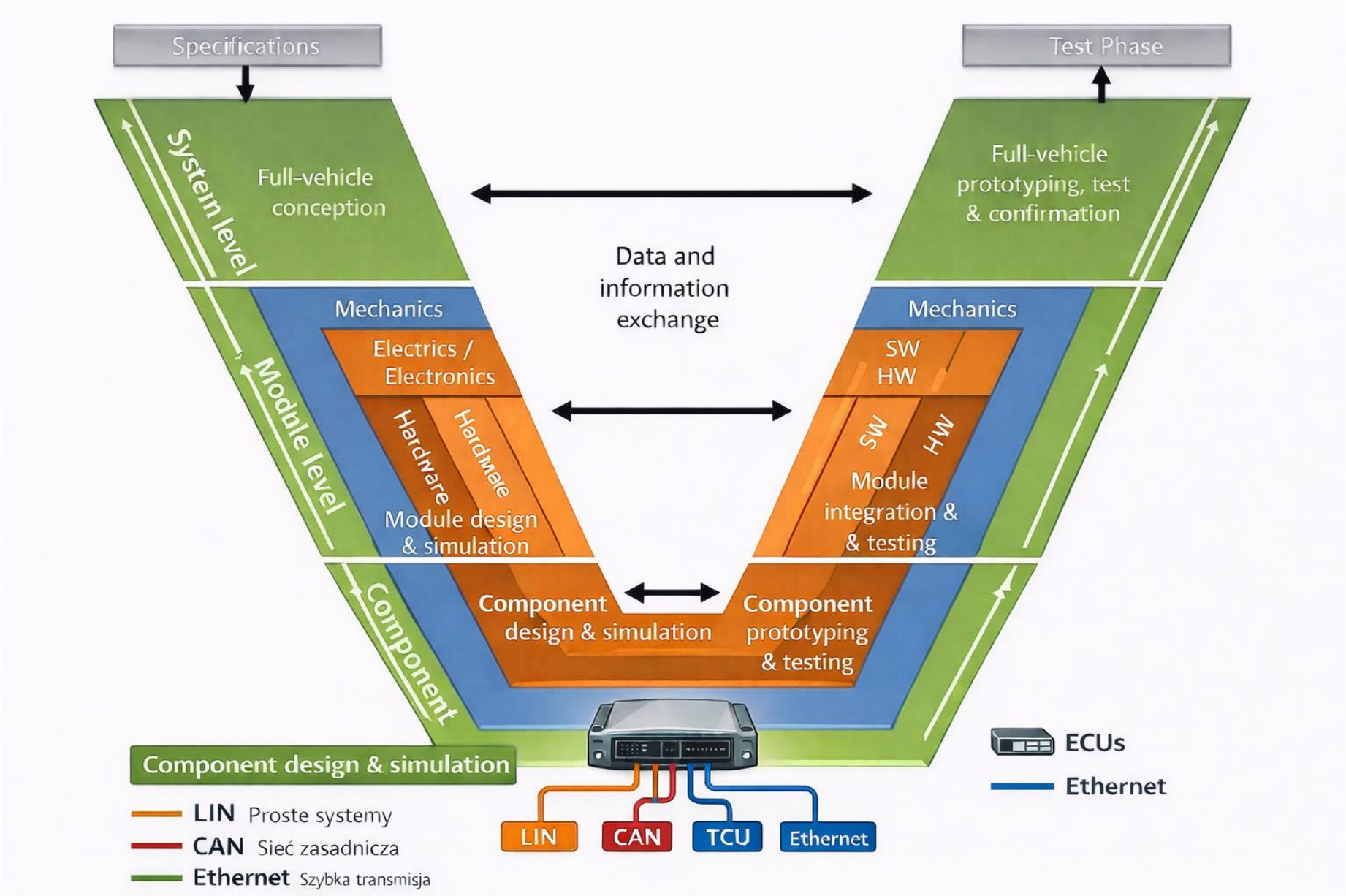
Github



Python

V-model pokazuje zależność między tworzeniem systemu a jego testowaniem.

Lewa strona V → co i jak projektujemy



Prawa strona V → jak sprawdzamy, że to działa

Lewa strona V – wymagania i projekt

1. Wymagania systemowe

Co ma robić cały system

Język „biznesowo-funkcjonalny”

np. System musi wykrywać poślizg i stabilizować tor jazdy

2. Wymagania architektury

Podział na ECU

Magistrale (CAN / LIN / Ethernet)

np. ESP ECU komunikuje się z Engine ECU po CAN.

3. Wymagania software

Logika, algorytmy

Stany, reakcje, timingi

np. Gdy poślizg > X, moment silnika ma zostać ograniczony w czasie < 10 ms.

4. Implementacja

Kod

konfiguracja

kalibracja

Prawa strona V – testy

1. Testy jednostkowe

↔ Implementacja

funkcje

moduły

algorytmy

2. Testy integracyjne

↔ Wymagania software

współpraca modułów

komunikacja CAN/LIN

3. Testy systemowe

↔ Wymagania systemowe

zachowanie całego ECU /

auta

scenariusze użytkownika

4. Testy akceptacyjne

↔ wymagania klienta /

OEM

„czy to jest to, co zamówił klient?”

Verification (lewa strona V → prawa środkowa)	Validation (górną prawą V)
code review static analysis unit test integration test requirement-based test Dowód: ✓ działa zgodnie z wymaganiem	vehicle test track test customer scenario test usability / drivability Dowód: ✓ działa zgodnie z oczekiwaniem użytkownika

	Verification	Validation
Pytanie	Czy dobrze zrobiliśmy?	Czy zrobiliśmy właściwą rzecz?
Porównanie	Specyfikacja	Real world / user need
Gdzie w V	środek prawej	top prawej
ASPICE	SWE.5, SWE.6, SYS.4	SYS.5

✓ Verification – „czy robimy to DOBRZE?”

- ✓ zgodnie z wymaganiami?
- ✓ poprawna implementacja?
- ✓ testy techniczne

np. Czy moment silnika jest ograniczany w <10 ms?

🎯 Validation – „czy robimy WŁAŚCIWĄ rzecz?”

czy system ma sens dla użytkownika?
czy spełnia cel biznesowy?

np. Czy kierowca faktycznie odzyskuje kontrolę nad autem?

🔗 Traceability = Requirement ↔ Test
Każde wymaganie:

- ma ID
- ma jeden lub więcej testów
- test wprost odnosi się do ID wymagania

Wymaganie:

REQ_SYS_123:

System musi ograniczyć moment silnika w czasie < 10 ms.

Test:

TC_SYS_045:

Sprawdzenie czasu reakcji ograniczenia momentu.

Covers: REQ_SYS_123

📖 **ASPICE to branżowy model oceny dojrzałości procesów system i software engineering w automotive, używany do zapewnienia jakości, traceability i powtarzalności developmentu.**

ASPICE (Automotive SPICE) to model oceny i doskonalenia procesów inżynierskich używany w branży automotive – głównie do oceny jakości procesów tworzenia systemów i oprogramowania w pojazdach (np. ECU).

ASPICE – Automotive Software Process Improvement and Capability Determination.

Co dokładnie opisuje ASPICE

1. Procesy (CO robisz?), np.

SYS - System engineering.

SWE - Software engineering.

SUP – Supporting processes [config, QA, change management].

MAN – Management [project, risk].

SYS.1 — Stakeholder Requirements	SWE.1 — SW Requirements
SYS.2 — System Requirements	SWE.2 — SW Architecture
SYS.3 — System Architecture	SWE.3 — Detailed Design
SYS.4 — System Integration & Test	SWE.4 — Unit Implementation
SYS.5 — System Qualification Test	SWE.5 — Unit Verification
	SWE.6 — Integration & Test

2. Capability Levels

Level	Znaczenie
0	brak procesu
1	proces działa
2	proces zarządzany
3	proces zdefiniowany i standardowy
4	proces mierzalny i kontrolowany
5	ciągłe doskonalenie

ISO 26262 to międzynarodowa norma dotycząca bezpieczeństwa funkcjonalnego systemów elektrycznych i elektronicznych w pojazdach drogowych. W praktyce: to fundament bezpieczeństwa dla nowoczesnych aut – zwłaszcza tych pełnych elektroniki, software'u i systemów ADAS 🚗💻

Najważniejsze pojęcia (must-know)

1. Bezpieczeństwo funkcjonalne

Chodzi nie o to, czy system działa, tylko czy zachowa się bezpiecznie, gdy przestanie działać poprawnie.

Przykład:

- ABS może się zepsuć
- ale nie może spowodować niekontrolowanego hamowania

2. Hazard i analiza ryzyka (HARA - Hazard Analysis and Risk Assessment)

Norma wymaga identyfikacji:

hazardów (potencjalnych zagrożeń)

ich ryzyka, ocenianego przez:

- S – Severity (ciężkość skutków)
- E – Exposure (częstotliwość sytuacji)
- C – Controllability (możliwość opanowania przez kierowcę)

FuSa (Functional Safety) w automotive oznacza **bezpieczeństwo systemów elektrycznych i elektronicznych E\E w pojeździe** — czyli zapewnienie, że system działa poprawnie albo przechodzi do bezpiecznego stanu, nawet gdy wystąpi awaria.

Przykład:

🔧 Jeśli zawiedzie czujnik pedału hamulca → system musi to wykryć → i np. ograniczyć moc silnika lub włączyć tryb awaryjny.

FuSa w automotive jest głównie zdefiniowana przez standard ISO 26262, który:

- opisuje proces projektowania bezpiecznych systemów,
- wprowadza poziomy ryzyka ASIL (A–D),
- określa wymagania dla hardware i software.

ASIL A → niskie ryzyko

ASIL D → najwyższe ryzyko

1. Koncept bezpieczeństwa (Concept Phase)
Analiza zagrożeń (Hazard Analysis, HARA)
Klasyfikacja ryzyka → ASIL A–D
Określenie wymagań bezpieczeństwa funkcjonalnego (Safety Goals)

2. Projekt systemu (System Level)
Specyfikacja wymagań systemowych (System Safety Requirements)
Definicja architektury systemu E/E
Określenie redundancji i mechanizmów monitorujących

3. Projekt hardware / software (Hardware & Software Development)
HW Design: wybór komponentów, analiza PFH, diagnostyka
SW Design: modelowanie, algorytmy, testy jednostkowe, integracja

4. Integracja i testy systemowe (Integration & Testing)
Integracja ECU i systemu
Testy funkcjonalne i bezpieczeństwa
Symulacje i testy w warunkach rzeczywistych
Walidacja zgodności z Safety Goals

5. Produkcja i eksploatacja (Production & Operation)
Procedury wytwarzania zgodne z FUSA
Monitoring działania systemu w pojeździe
Update oprogramowania (jeśli wymagany)
Analiza wypadków / incydentów (Safety Case)

6. Dezaktywacja / utylizacja (Decommissioning)
Bezpieczne wyłączenie systemu
Przechowywanie danych diagnostycznych
Analiza końcowa z punktu widzenia FUSA

3. ASIL – [Automotive Safety Integrity Level] Poziom krytyczności bezpieczeństwa:

ASIL A – Najniższy poziom ryzyka. Wymaga minimalnych środków bezpieczeństwa (np. oświetlenie wnętrza, tylne światła).

ASIL B – Umiarkowany poziom ryzyka. Wymaga podstawowych mechanizmów bezpieczeństwa (np. tempomat, asystent świateł drogowych).

ASIL C – Wysoki poziom ryzyka. Wymaga zaawansowanych procedur i rygorystycznych środków (np. aktywne sterowanie, niektóre funkcje ABS).

ASIL D – Najwyższy poziom ryzyka (najbardziej krytyczny). Wymaga najwyższego rygoru, weryfikacji i redundancji, ponieważ awaria zagraża życiu (np. poduszki powietrzne, układ hamulcowy ABS/ESP, wspomaganie kierownicy).

Co dokładnie obejmuje ISO 26262?

Norma jest podzielona na części, m.in.:

Zarządzanie bezpieczeństwem

Koncepcja bezpieczeństwa

Rozwój systemu

Rozwój sprzętu

Rozwój oprogramowania

Produkcja i eksploatacja

Wsparcie (narzędzia, kwalifikacje, dokumentacja)

Czyli: hardware + software + procesy + ludzie.

Przykład z życia

Elektroniczne wspomaganie kierownicy (EPS):

hazard: nagły skręt kierownicy

ASIL: zwykle C lub D

wymagania:

redundancja czujników

monitorowanie błędów

tryb degradacji (fail-safe / limp mode)

rygorystyczne testy software'u

Jeśli ktoś pracuje w automotive i nie zna ISO 26262, to prędzej czy później... i tak będzie musiał 😊

W kontekście Automotive, Safety Lifecycle (Cykl życia bezpieczeństwa) to ustrukturyzowany proces zdefiniowany w normie ISO 26262, który ma na celu wyeliminowanie lub zminimalizowanie ryzyka związanego z awariami systemów elektrycznych i elektronicznych (E/E) w pojazdach.

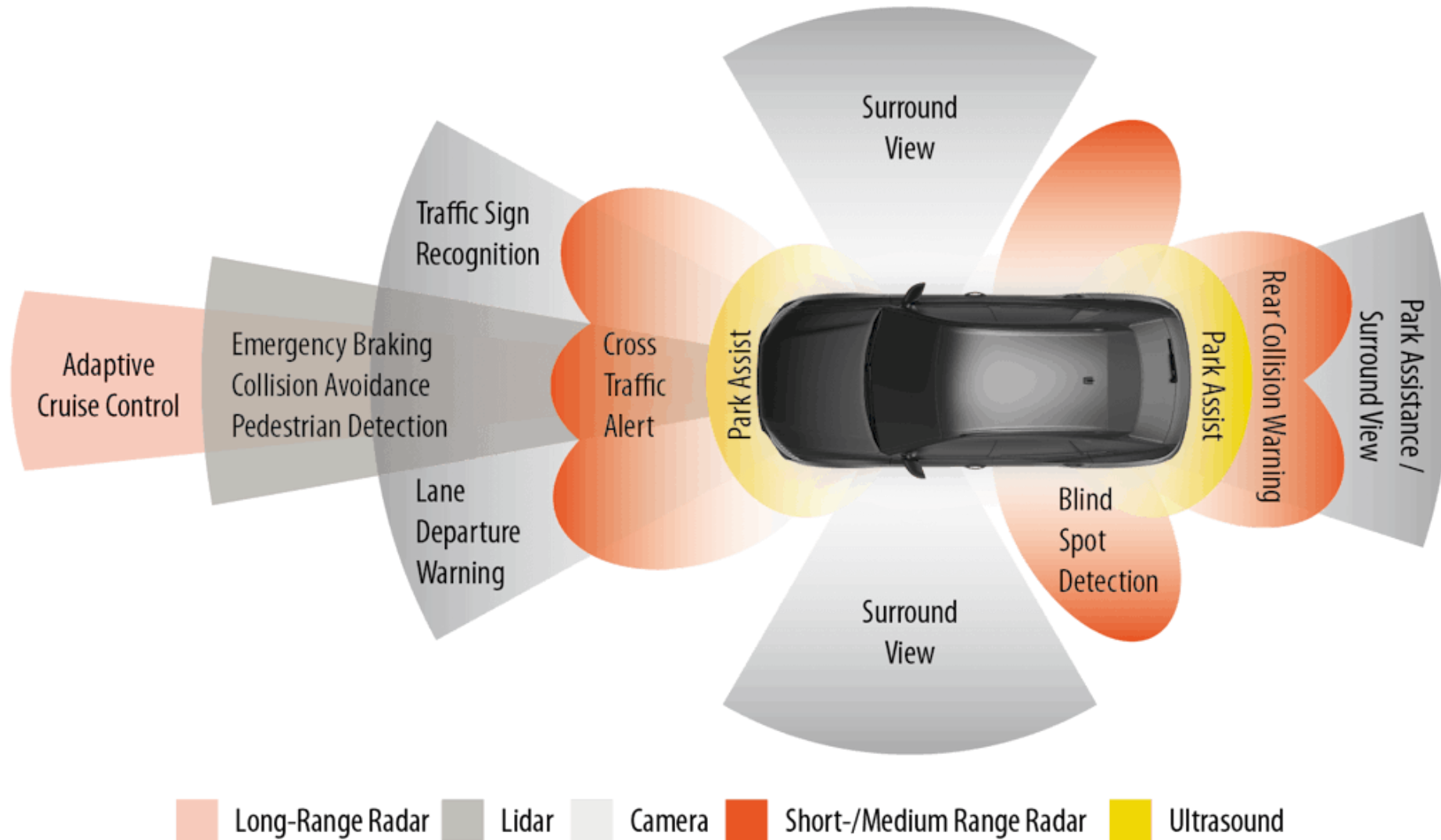
Proces ten obejmuje cały „życiowy” produktu – od pierwszej idei aż po jego wycofanie z użytku.

1. Faza koncepcyjna (Concept Phase) To tutaj zapadają kluczowe decyzje dotyczące bezpieczeństwa: Definicja elementu (Item Definition): Opisanie, co system ma robić i jak wchodzi w interakcję z innymi. HARA (Hazard Analysis and Risk Assessment): Identyfikacja potencjalnych zagrożeń (np. nagłe zablokowanie kół) i przypisanie im poziomów ASIL (A-D). Cele bezpieczeństwa (Safety Goals): Sformułowanie ogólnych wymagań, które zapobiegają zidentyfikowanym zagrożeniom.

2. Rozwój produktu (Product Development) Najbardziej rozbudowana faza, realizowana zgodnie z modelem V: Poziom systemowy: Tworzenie architektury systemu i określanie technicznych wymagań bezpieczeństwa. Poziom sprzętowy (Hardware): Projektowanie i testowanie podzespołów (np. procesorów, czujników) pod kątem awarii losowych. Poziom programowy (Software): Pisanie kodu zgodnie z rygorystycznymi standardami, aby uniknąć błędów projektowych. Integracja i walidacja: Sprawdzenie, czy gotowy produkt rzeczywiście spełnia założone cele bezpieczeństwa w rzeczywistych warunkach.

3. Produkcja i eksploatacja (Production & Operation) Bezpieczeństwo musi być zachowane również po opuszczeniu fabryki: Produkcja: Zapewnienie, że każdy egzemplarz jest wykonany poprawnie i bez wad zagrażających bezpieczeństwu. Obsługa i serwis: Procedury napraw, aby nie naruszyć integralności systemów bezpieczeństwa. Wycofanie z eksploatacji (Decommissioning): Bezpieczne usunięcie produktu z rynku.

ADAS – Advanced Driver Assistance Systems



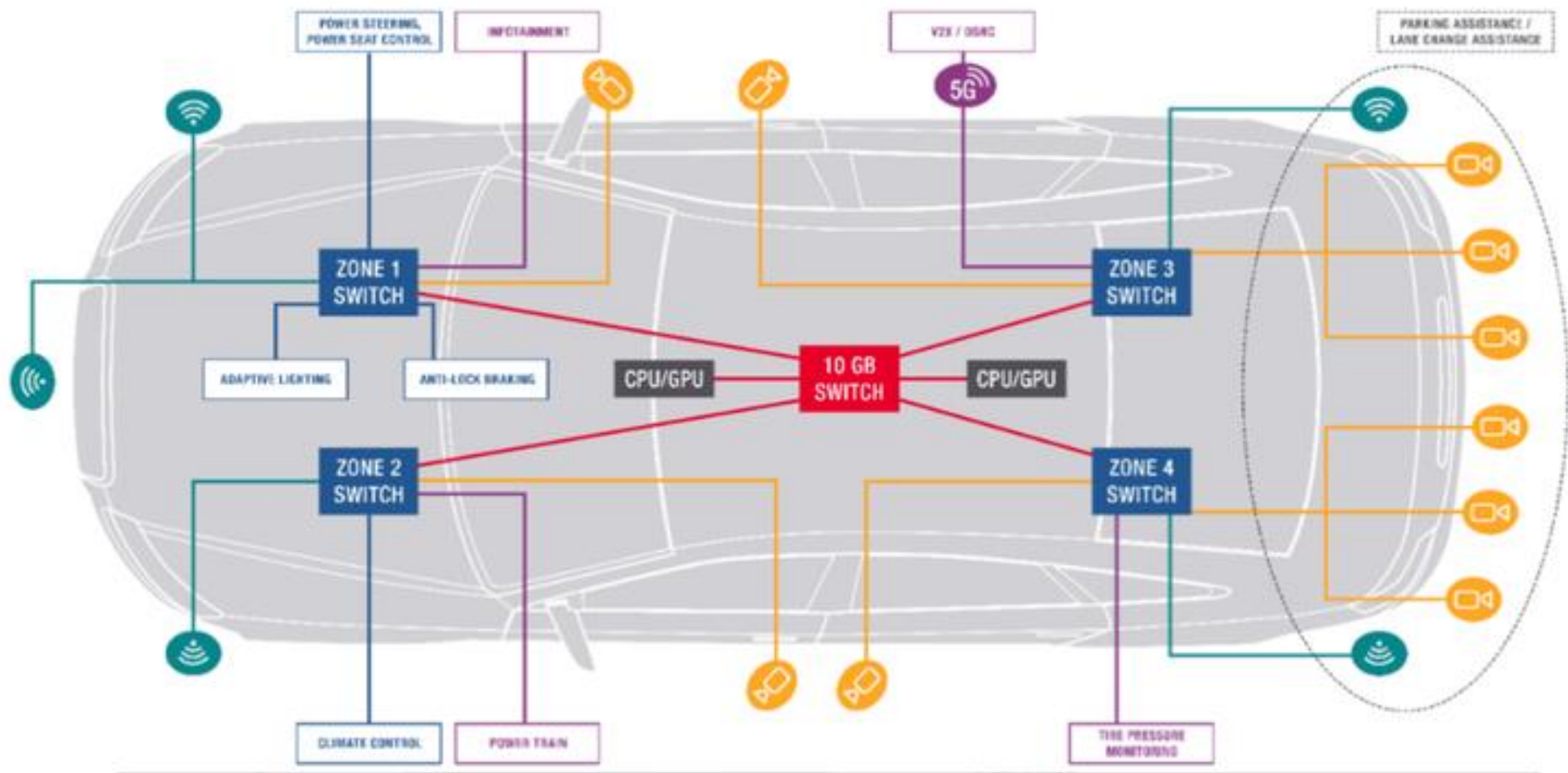
ADAS to zbiór systemów elektronicznych w pojazdach, które wspierają kierowcę w prowadzeniu samochodu z myślą o poprawie bezpieczeństwa i komfortu. To technologie, które pomagają wykrywać potencjalne zagrożenia, ostrzegać kierowcę, a w niektórych przypadkach interweniować (hamowanie, korekta toru) zanim kierowca zareaguje.

Z czego składa się ADAS – podstawowe komponenty

- Czujniki i urządzenia wejściowe: kamery, radary, lidar, ultradźwięki – służą do wykrywania otoczenia pojazdu.
- Jednostka przetwarzająca / sterownik (ECU): zbiera dane z czujników, analizuje je algorytmami, decyduje o ewentualnej akcji.
- Aktuatory / mechanizmy wykonawcze: systemy pojazdu (hamulce, kierownica, systemy wspomagania) które mogą podjąć działanie lub wspomóc kierowcę.
- Interfejs z kierowcą: ostrzeżenia dźwiękowe, wizualne albo haptyczne (np. wibracje) – informujące kierowcę lub proszące o interwencję.

Przykłady typowych funkcji wchodzących w zakres ADAS

- Ostrzeganie o niezamierzonej zmianie pasa ruchu (LDW – Lane Departure Warning) i/lub wspomaganie utrzymania pasa (Lane Keep Assist).
- Adaptacyjny tempomat (ACC – Adaptive Cruise Control) – utrzymuje bezpieczny odstęp od pojazdu poprzedzającego.
- Automatyczne hamowanie awaryjne (AEB – Autonomous Emergency Braking) – w sytuacji potencjalnej kolizji system sam hamuje lub wspomaga hamowanie.
- Wykrywanie pojazdu w martwym polu (Blind Spot Detection), ostrzeganie przy cofaniu lub przy skręcie w tył drogi (Cross-Traffic Alert).
- Systemy wspomagania parkowania, sterowania światłami (adaptacyjne światła), rozpoznawania znaków drogowych.



LEGEND



Sensor Fusion (Fuzja czujników) - to proces łączenia danych z różnych typów czujników (radarów, kamer, LiDAR-ów, czujników ultradźwiękowych), aby uzyskać obraz otoczenia bardziej wiarygodny i dokładny, niż byłoby to możliwe przy użyciu tylko jednego z nich.

Dlaczego jest konieczna?

Każdy czujnik ma swoje słabości. Kamera świetnie rozpoznaje kolory i znaki, ale gubi się w gęstej mgle. Radar doskonale mierzy odległość i prędkość nawet w nocy, ale nie „widzi” linii na asfalcie.

Jak to działa?

Algorytmy (np. Filtr Kalmana) nakładają na siebie dane, eliminując szумы i niepewności. Wynikiem jest tzw. **World Model (model świata)** – cyfrowa mapa wszystkiego, co dzieje się wokół auta.

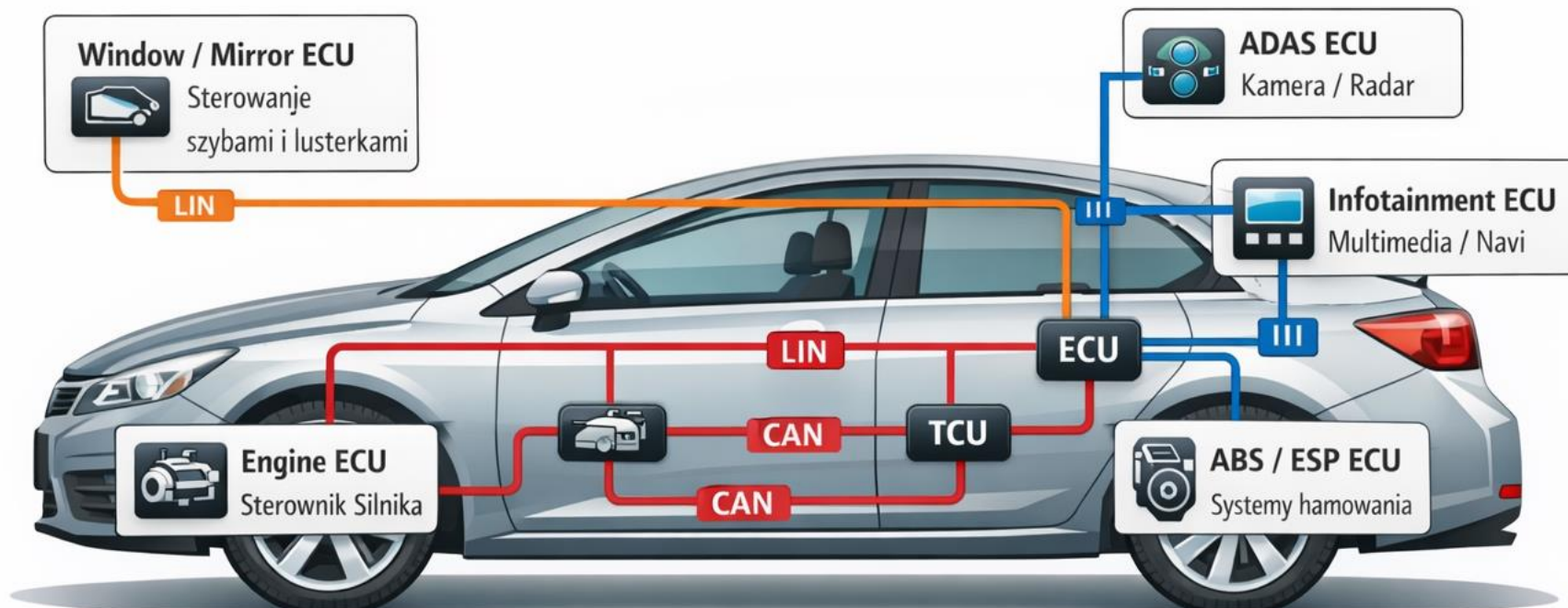
Etap	Odpowiednik ludzki	Przykładowa operacja
Sensor Fusion	Zmysły (Wzrok + Słuch)	łączenie obrazu z kamery z echem radaru.
Perception	Rozpoznawanie	"To jest rowerzysta na moim torze jazdy."
Decision	Myślenie / Wybór	"Muszę zahamować, żeby uniknąć kolizji."
Control	Ruch mięśni	Wysłanie sygnału do zacisków hamulcowych.

Ten łańcuch musi działać się w czasie rzeczywistym (często w milisekundach).

Architektura: Perception → Decision → Control To klasyczny rurociąg (pipeline) przetwarzania danych w systemach ADAS. Można go porównać do działania ludzkiego organizmu.

- I. Perception (Percepcja – „Widzę”)** - To etap, w którym surowe dane z czujników (po fuzji) są interpretowane. System musi odpowiedzieć na pytanie: Co znajduje się wokół mnie? Wykrywanie obiektów: Czy to, co widzę, to inny samochód, pieszy czy barierka? Klasyfikacja i śledzenie: Ten pieszy porusza się w stronę jezdni z prędkością 5 km/h. Lokalizacja: Gdzie dokładnie na pasie ruchu znajduje się mój samochód?
- II. Decision / Planning (Decyzja i Planowanie – „Myślę”)** - Na podstawie modelu świata z etapu percepcji, system musi podjąć decyzję o dalszym działaniu. Musi odpowiedzieć na pytanie: Co powinienem zrobić? Prediction (Przewidywanie): Czy ten samochód z lewej zajedzie mi drogę? Path Planning (Planowanie ścieżki): Wyznaczenie bezpiecznej trasy ominięcia przeszkody lub utrzymania pasa. Behaviors (Zachowania): Podjęcie decyzji: „Hamuj awaryjnie”, „Zwolnij”, „Zmień pas”.
- III. Control (Sterowanie – „Robię”)** - To ostatni etap, w którym decyzja zostaje zamieniona na konkretne sygnały elektryczne wysyłane do elementów wykonawczych pojazdu (tzw. actuators). Musi odpowiedzieć na pytanie: Jak fizycznie wykonać manewr? Steering: Skręcenie kierownicy o odpowiedni kąt. Acceleration/Braking: Zwiększenie ciśnienia w układzie hamulcowym lub zmiana momentu obrotowego silnika. Feedback loop: System stale sprawdza, czy wykonany ruch przyniósł zamierzony efekt (np. czy auto faktycznie zwolniło).

— **LIN** Proste systemy — **CAN** Zasadnicza sieć — **Ethernet** Szybka transmisja



 **LIN** Proste systemy

 **CAN** Sieć zasadnicza

 **Ethernet** Szybka transmisja
1 Gb/s

Co to jest ECU? - Electronic Control Unit (Elektroniczny sterownik)

- zbiera dane z czujników
- przetwarza je
- steruje elementami wykonawczymi

W nowoczesnym aucie jest od kilkunastu do nawet kilkudziesięciu ECU.

Przykłady ECU w samochodzie
ECM / PCM – sterownik silnika
TCU – sterownik skrzyni biegów
ABS / ESP ECU – układ hamulcowy
BCM – nadwozie (światła, zamki)
ADAS ECU – kamery, radar
Infotainment ECU – multimedia

W środku:
mikrokontroler / SoC
pamięć Flash, RAM, EEPROM
interfejsy: CAN, LIN, FlexRay, Ethernet
ECU:
wykonuje algorytmy sterowania
komunikuje się z innymi ECU
obsługuje diagnostykę (np. UDS)

ECU a diagnostyka
Z poziomu diagnostyki możesz:
odczytać DTC (błędy)
podejrzeć dane bieżące
wykonać testy i adaptacje
programować / kodować ECU

**ECU = mózg jednego systemu w aucie
a całe auto to sieć wielu ECU gadających ze sobą.**

UDS Unified Diagnostic Services to protokół diagnostyczny, którym tester diagnostyczny (OBD)komunikuje się z ECU

Odczytuje błędy DTC – Diagnostic Trouble Code. Odczytuje dane bieżące Wykonuje testy i adaptacje Programuje flash ECU (Electronic Control Unit)	Client – Server Client → tester diagnostyczny Server → ECU Client zawsze inicjuje komunikację.
0x10 – Diagnostic Session Control Default (0x01) - normalna jazda Extended (0x03) - diagnostyka Programming (0x02) – flashowanie	0x11 – ECU Reset (Restart Sterownika) hard reset key off/on reset
0x22 – Read Data By Identifier (DID) Odczyt numeru VIN (DID: 0xF1 90) Odczyt parametrów silnika (np. DID: 0x01 05) Odczyt wersji oprogramowania (DID: 0xF1 95) Odczyt stanu naładowania akumulatora (SOC)	0x19 – Read DTC Information 01 Odczyt ilości błędów DTC 02 Odczyt błędów wraz z opisem 04 Odczytać Freeze Frame – parametrów pojazdu w chwili wystąpienia błędu.
0x34 / 0x36 / 0x37 – Flashowanie ECU Sekwencja programowania: Request Download (0x34) Transfer Data (0x36) Transfer Exit (0x37)	Odpowiedzi Negatywne (NRC) 0x10 General Reject 0x11 Service Not Supported 0x13 Incorrect Message Length 0x33 Security Access Denied

Czym różni się programowanie od kodowania ECU?

<i>Czynność</i>	<i>Co robi</i>	<i>Przykład</i>
Programowanie ECU	Wgrywa firmware	nowa wersja sterownika
Kodowanie ECU	Zmienia konfigurację	halogeny vs LED
Adaptacja	Uczy ECU	adaptacja skrzyni
Diagnostyka	Odczyt DTC	błąd czujnika

Wszystko to zwykle idzie przez **UDS** → protokół do diagnostyki i programowania

LIN (Local Interconnect Network) - Najprostsza i najtańsza magistrala w aucie

•sterowanie szybami •lusterka •czujniki deszczu •klimatyzacja (elementy wykonawcze)	Działanie: •Master–Slave (jeden sterownik rządzi, reszta odpowiada) •prędkość: do ~20 kb/s •komunikacja po jednym przewodzie
---	---

PLUSY ✓ tania ✓ prosta	MINUSY ✗ bardzo wolna ✗ brak redundancji
---	---

FlexRay - Szybka, deterministyczna i „bezpieczna” - Systemy wymagające **precyzyjnego czasu** i niezawodności.

steering-by-wire brake-by-wire aktywne zawieszenie ADAS (starsze generacje)	Time-triggered (komunikacja w slotach czasowych) 2 kanały → redundancja prędkość: do 10 Mb/s
PLUSY ✓ deterministyczna (wiemy <i>dokładnie</i>, kiedy co przyjdzie) ✓ bardzo niezawodna	MINUSY ✗ droga ✗ skomplikowana konfiguracja

Automotive Ethernet - Nowoczesna sieć „jak w IT”, ale do Auta

kamery radar / lidar infotainment ADAS / autonomous driving	Działanie: pakiety (jak TCP/IP, UDP) 100 Mb/s 1 Gb/s nawet 10 Gb/s topologia gwiazdy / switche
PLUSY ✓ bardzo szybki ✓ skalowalny ✓ jeden standard na wszystko	MINUSY ✗ większa złożoność ✗ trzeba ogarniać bezpieczeństwo (cybersecurity)

Porównanie

Magistrala	Szybkość	Zastosowanie	Styl
LIN	~20 kb/s	Proste elementy	tania & wolna
FlexRay	10 Mb/s	Safety-critical	deterministyczna
Eth	100 Mb/s – Gb/s	Kamery, ADAS	nowoczesna

C A N

Controller Area Network

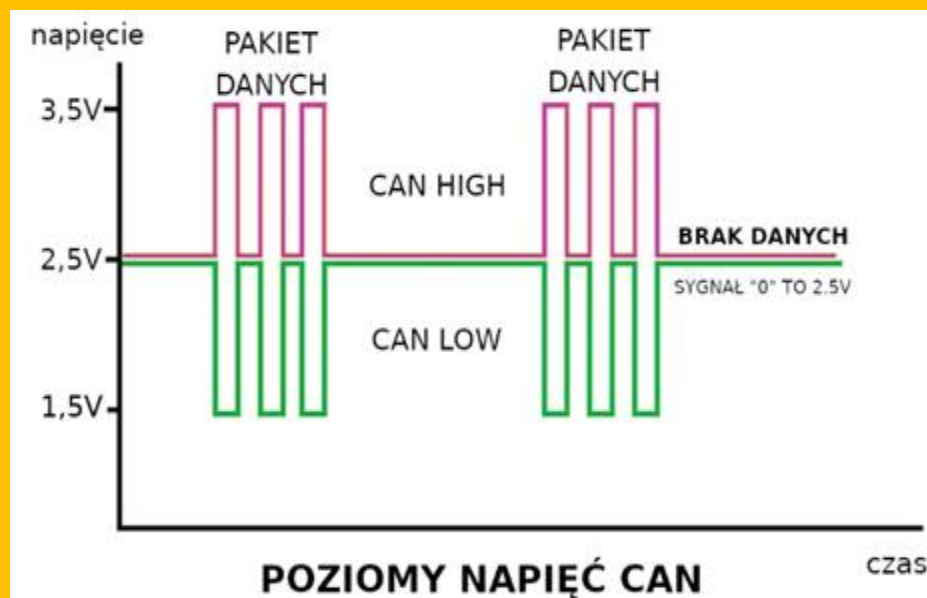
Najpopularniejszy standard (od lat 90.), stosowany m.in. w ECU (elctronic control unit), ABS, poduszkach powietrznych, silniku.

- **Classical CAN** (do 1 Mbps),
- **CAN FD (Flexible Data Rate)** (do 8 Mbps).

Sieć informatyczna, która pozwala na szybką wymianę pakietów danych w czasie rzeczywistym.

Szyna CAN to dwa przewody (pin 2 oraz pin 7) przesyłające te same, ale odwrócone w fazie o 180 stopni pakiety danych.

W przypadku CAN poziomy napięcie dla HIGH to 3.5V, a LOW to 1.5V. Jak widać na wykresie skok dla pojedynczej linii to 1V.



liczy się różnica napięć (~2 V) - absolutne poziomy mogą się różnić zależnie od transceivera !!!

Pochodne CAN

CAN 2.0 jest to podstawowa wersja protokołu CAN, określona przez standard ISO 11898-1. Obejmuje dwie główne wersje: CAN 2.0A i CAN 2.0B. Wersja 2.0A używa identyfikatorów 11-bitowych, podczas gdy wersja 2.0B wykorzystuje identyfikatory 29-bitowe. Obie wersje działają w prędkościach transmisji od 10 kbit/s do 1 Mbit/s.

CAN FD (Flexible Data-Rate), jest rozwinięciem standardu CAN, który wprowadza możliwość transmisji danych w wyższych prędkościach. CAN FD pozwala na większe szybkości transmisji danych, co jest szczególnie korzystne w aplikacjach o dużym obciążeniu sieci lub w sytuacjach, gdy wymagane jest szybsze przesyłanie danych.

CANopen jest to protokół warstwy aplikacyjnej oparty na standardzie CAN, stosowany głównie w przemyśle, w tym w sterowaniu maszynami i urządzeniami. CANopen definiuje zestaw standardowych wiadomości i usług, które umożliwiają komunikację między różnymi urządzeniami w sieci CAN.

DeviceNet jest to inny protokół warstwy aplikacyjnej oparty na standardzie CAN. Został zaprojektowany z myślą o przemyśle, zwłaszcza w zastosowaniach związanych z automatyką przemysłową, sensorami i aktywatorami.

J1939 jest to protokół wykorzystywany w przemyśle motoryzacyjnym, szczególnie w ciężarówkach i pojazdach ciężkich. J1939 definiuje komunikację między różnymi komponentami pojazdu, takimi jak silniki, skrzynie biegów, układy hamulcowe itp.

UART – Universal Asynchronous Receiver/Transmitter

Definicja i zasada działania

UART to sprzętowy mechanizm/protokół służący do **asynchronicznej komunikacji szeregowej** pomiędzy dwoma urządzeniami — nadajnikiem (TX) i odbiornikiem (RX).

„Asynchroniczna” oznacza tutaj, że **nie ma dodatkowej linii zegara** przesyłającej sygnał synchronizacji między nadajnikiem a odbiornikiem. Oba urządzenia muszą być ustawione na ten sam **baud rate** (liczba bitów na sekundę).

W skrócie: nadajnik przekształca dane równoległe (np. bajt) na strumień bitów, przesyła je po jednej linii (lub dwóch – TX i RX), a odbiornik ponownie rekonstruuje bajt z przesłanych bitów.

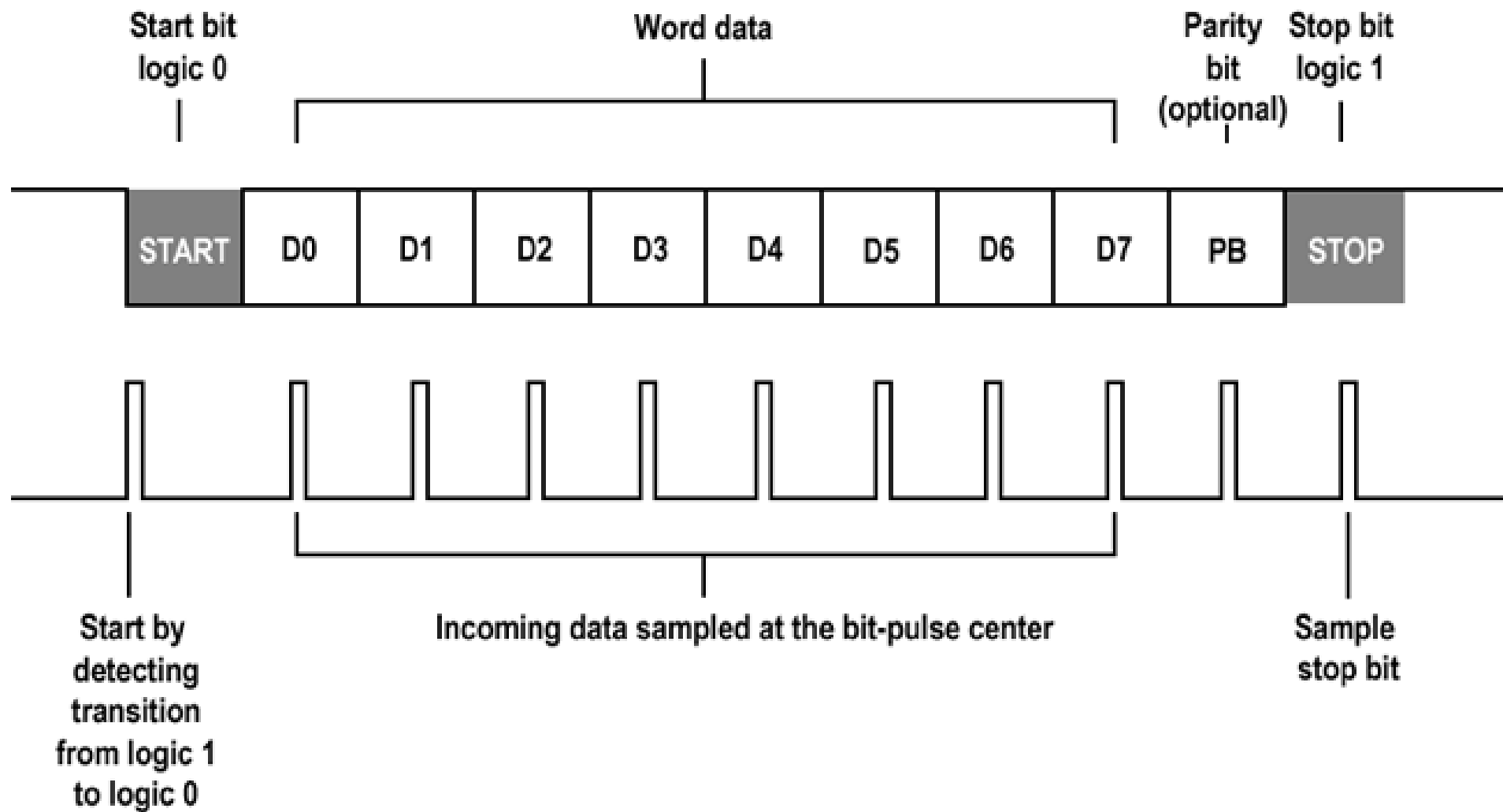
Główne cechy i właściwości protokołu

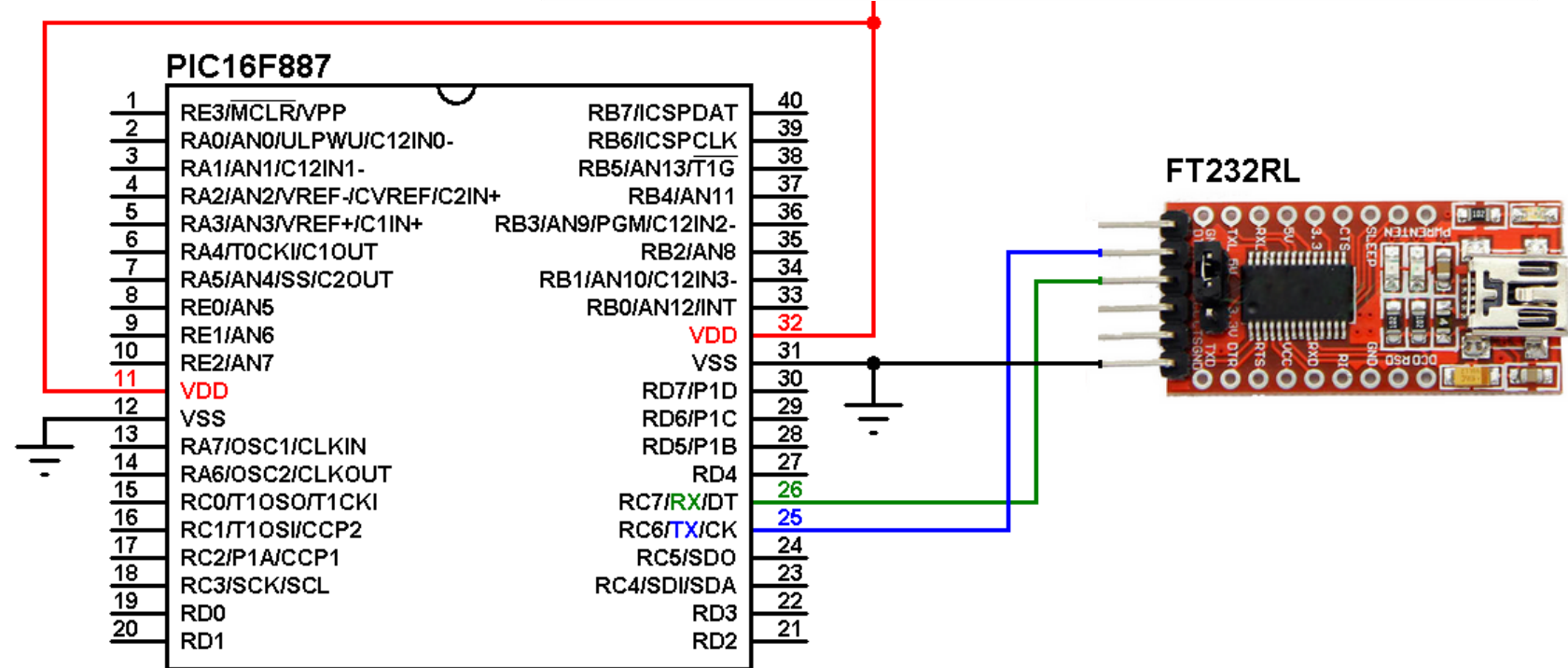
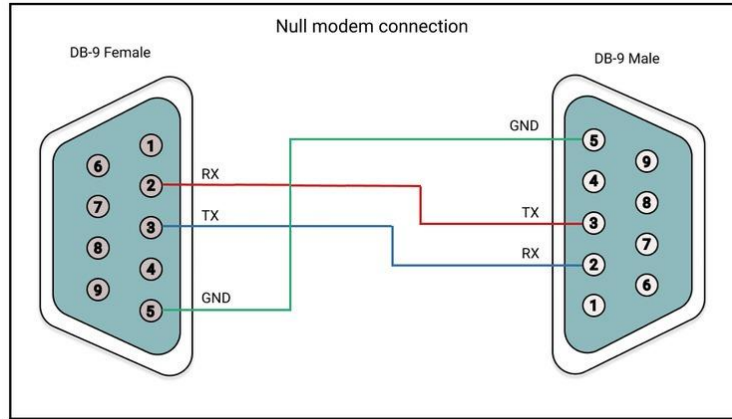
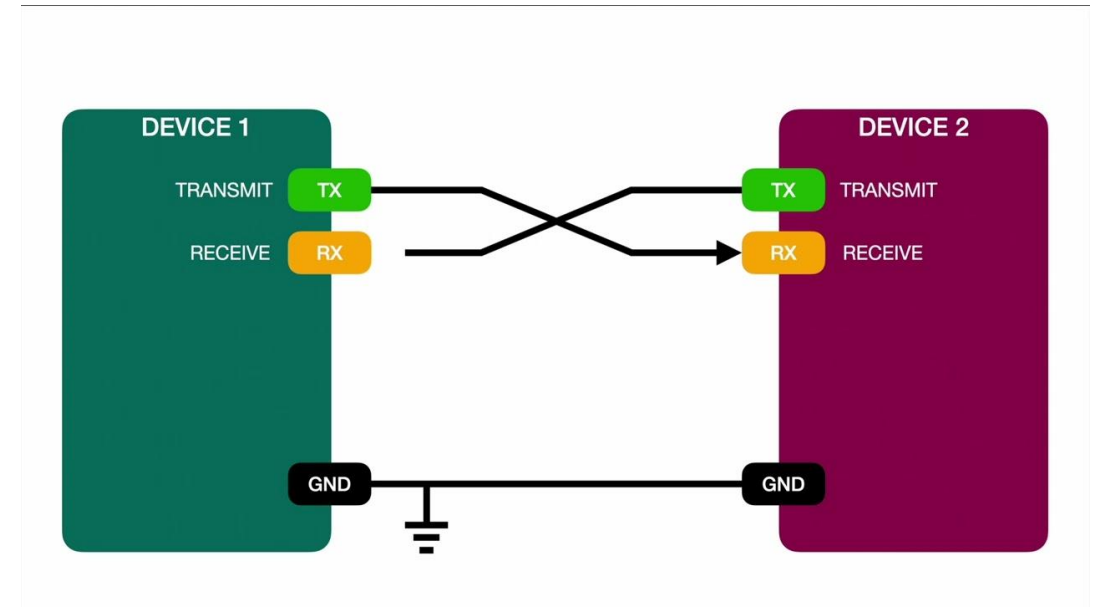
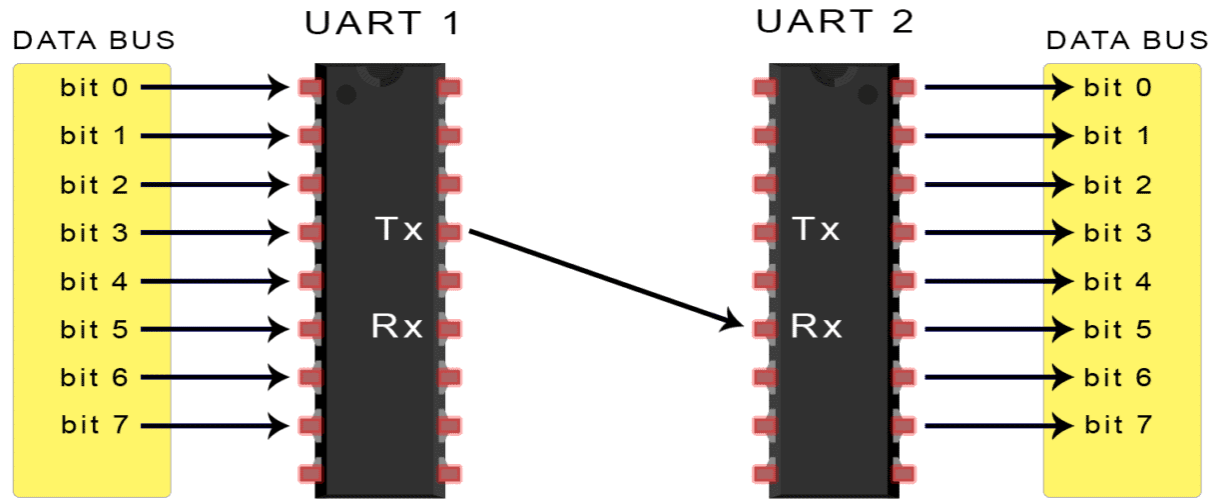
Minimalna liczba przewodów: typowo dwa sygnały — TX i RX (plus wspólna masa).

Brak zegara przesyłanego: synchronizacja odbywa się na podstawie bitu startowego i ustalonego baud rate.

Format ramki danych, przykładowo:

Bit startu: linia przechodzi z poziomu „idle” (zwykle wysoki) na niski, co sygnalizuje początek transmisji.





- **Typowe zastosowania**
- Komunikacja pomiędzy mikroprocesorem/mikrokontrolerem a komputerem lub układem pomocniczym (debugowanie, logi).
- Proste połączenia szeregowe w systemach wbudowanych (embedded): czujniki, moduły komunikacyjne, interfejsy konwerterów TTL-RS232 itp.
- Porty szeregowe w starszych komputerach, terminalach (np. RS-232 bazującym na UART) — choć dziś częściej używane w formie TTL lub USB-to-UART.
- **Wady i zalety w porównaniu do innych protokołów (np. SPI, I²C)**
- **Zalety UART:**
 - Bardzo prosty do zaimplementowania — tylko kilka parametrów do ustawienia.
 - Niski koszt sprzętowy (niewiele przewodów, zazwyczaj jedna linia TX + RX).
 - Popularność i szerokie wsparcie w mikrokontrolerach i urządzeniach.
- **Wady UART:**
 - Brak wspólnego zegara — synchronizacja może być mniej stabilna niż w protokołach synchronicznych.
 - Prędkość przesyłu ograniczona — zazwyczaj mniejsza niż w SPI.
 - Z reguły komunikacja punkt-do-punkt (jeden nadajnik, jeden odbiornik) — trudniej o magistralę wielu urządzeń.
 - Ramkowanie danych i protokołów na wyższym poziomie (np. określenie początku i końca wiadomości) częściej trzeba zaimplementować samodzielnie.

Cecha	PROFINET	EtherNet/ IP	Modbus	Profibus	I ² C	RS232	RS485
Warstwa	Ethernet RT	Ethernet	Aplikacja	Fieldbus	Embedded	Fizyczna	Fizyczna
Prędkość	100M–1G	100M–1G	do 115 kb/s	do 12 Mb/s	do 1 Mb/s	do 115 kb/s	do 10 Mb/s
Czas cyklu	< 1 ms	2–20 ms	> 50 ms	determini st.	brak	brak	zależny
Zasięg	100 m	100 m	1200 m	1200 m	cm	15 m	1200 m
Diagnostyka	Bardzo wysoka	Wysoka	Niska	Średnia	Minimalna	Minimaln a	Fizyczna
Typowe użycie	Motion, safety	PLC Rockwell	Liczniki	Starsze linie	PCB	Serwis	Fieldbus